

CHAPTER-1

INTRODUCTION

1.1.BACKGROUND OF THE STUDY

Cardiovascular disease (CVD) remains one of the leading causes of death and disability worldwide. According to the World Health Organization (2023), nearly 17.9 million people die from CVD each year, accounting for 32% of all global deaths. Of these deaths, 85% are due to heart attacks and strokes. This makes CVD not only a medical challenge but also a serious socio-economic burden, particularly for low- and middle-income countries, which together account for more than three-quarters of these deaths. Rapid urbanization, sedentary lifestyles, unhealthy diets, stress, and tobacco and alcohol use have significantly increased the prevalence of heart-related conditions.

In India, the situation is even more concerning. India has seen a sharp rise in cases over a past few decades, with CVD now causing about 28% of all deaths (ICMR, 2021). Unlike in many developed nations where CVD primarily affects older populations, India faces a unique challenge—cardiovascular conditions are often seen in younger individuals, leading to the loss of productive years of life. Factors such as changing lifestyles, rising obesity, diabetes, hypertension, and lack of awareness further aggravate the problem.

Early detection and timely intervention are therefore critical in addressing this health crisis. Many cardiovascular conditions progress silently, showing little to no symptoms until they reach a severe stage. Regular screening, risk prediction, and preventive healthcare can significantly reduce complications, hospitalizations, and deaths. By identifying risks early, individuals can adopt healthier lifestyles, receive appropriate treatment, and prevent life-threatening events such as heart attacks and strokes.

Thus, focusing on early detection of cardiovascular disease is not just a medical necessity but also a social and economic imperative, especially in a country like India where the burden of heart disease is rising at an alarming rate.

1.2.PROBLEM DOMAIN

Cardiovascular disease (CVD) poses a serious health challenge because it often goes undetected until it has reached an advanced stage. Many individuals live with underlying risks such as hypertension, diabetes, high cholesterol, or obesity without realizing the danger, since the early symptoms are usually mild or absent. By the time CVD is diagnosed, patients may already be at risk of life-threatening complications such as heart attack, stroke, or heart failure.

Another major challenge lies in the methods of detection. Standard diagnostic tests — such as ECG, echocardiograms, or stress tests — require specialized equipment and clinical facilities. These are not always easily accessible, especially in rural or resource-limited areas. Even when available, patients may delay testing due to lack of awareness, financial constraints, or the inconvenience of hospital visits.

As a result, early detection and timely intervention become difficult, leading to higher hospitalization rates, costly treatments, and preventable deaths. There is an urgent need for more accessible, affordable, and proactive solutions that can predict cardiovascular risks before they reach a critical stage.

1.3.AIM OF THE PROJECT

The primary aim of this project is to develop a lightweight, web-based application that can predict an individual's risk of cardiovascular disease and provide personalized lifestyle recommendations. The system is designed to be simple, accessible, and effective, enabling users to assess their heart health without the need for immediate clinical tests.

By integrating machine learning models with user-friendly interfaces, the project seeks to:

- Predict cardiovascular risk based on user-provided health parameters.
- Promote early detection of potential heart-related issues before they become severe.

- Offer lifestyle guidance tailored to individual needs, such as diet, exercise, and preventive care.
- Increase accessibility by creating a platform that can be used anytime and anywhere, especially benefiting individuals in areas with limited healthcare resources.

Ultimately, the goal is not just to create a predictive tool, but to build a digital health companion that empowers users to take proactive steps in managing their heart health and reducing the risk of life-threatening conditions.

1.4.SCOPE OF THE PROJECT

This project is primarily aimed at serving as a screening and awareness tool for cardiovascular disease (CVD). It helps users understand their potential risk levels through simple inputs and provides actionable recommendations such as lifestyle improvements, preventive measures, and guidance on when to consult a medical professional. The intention is to create awareness and encourage early detection, not to replace doctors or clinical diagnosis.

At its current stage, the system operates without a database, ensuring a lightweight and privacy-friendly design. However, it has been built with scalability in mind, meaning it can be extended in the future to support advanced features such as:

- Maintaining patient history for better tracking and personalized insights
- Offering a multilingual interface to reach users across diverse linguistic backgrounds
- Expanding into a mobile application for easier accessibility and real-time usage
- Exporting and sharing results in professional formats such as PDF and DOCX, enabling patients to keep records or share them directly with clinicians.

As an awareness-oriented prototype, the project aims to empower individuals, promote preventive healthcare, and bridge the gap between patients and timely medical care.

1.5 CONTRIBUTIONS

This project stands out due to its unique blend of machine learning, healthcare awareness, and accessibility-focused design. Unlike many academic prototypes that remain theoretical, this work delivers a practical, working solution that can be expanded in the future. The key contributions include:

- **ML Model (care4heart.pkl):** A trained machine learning model capable of predicting cardiovascular disease risk based on user-provided health parameters. This forms the core intelligence of the system.
- **Flask Web Application:** A lightweight yet powerful backend that allows users to interact with the model in real time, making predictions quick, reliable, and easily accessible over the web.
- **Rule-Based Recommendation System:** Complements the ML predictions by offering context-aware lifestyle tips, doctor recommendations, and urgent steps for high-risk users, ensuring the system not only predicts risk but also guides users toward actionable next steps.
- **User-Friendly Interface:** A clean, simple, and intuitive UI specifically designed to be used by non-technical individuals, ensuring that awareness and preventive care are accessible to a wider population.

Together, these contributions demonstrate how AI, rules-based logic, and human-centered design can work hand-in-hand to build an effective screening and awareness tool for cardiovascular disease.

CHAPTER-2

PROBLEM STATEMENT

2.1.PROBLEM STATEMENT

Heart diseases remain one of the leading causes of mortality worldwide, with a rapidly increasing burden in India. While traditional diagnostic methods and health check-ups are effective, they are often costly, time-consuming, and inaccessible to a large portion of the population. Many individuals—particularly in rural and semi-urban areas—face barriers such as financial constraints, lack of awareness, and limited healthcare infrastructure, which leads to delays in detection.

This late diagnosis often results in advanced disease progression, higher treatment costs, loss of productive years, and greater socio-economic burden. Despite advancements in medical technology, there is still a lack of **affordable, easy-to-use, and awareness-driven digital tools** that can help individuals identify their cardiovascular risk early.

Therefore, the core problem addressed in this project is:

- Limited accessibility of affordable and timely cardiovascular screening.
- Lack of awareness-oriented tools for preventive healthcare.
- Absence of simple digital solutions that empower individuals to take proactive steps before complications arise.

The project seeks to bridge this gap by developing a **machine learning–powered digital tool** that enables early cardiovascular risk assessment, promotes awareness, and encourages timely medical consultation.

2.2.OBJECTIVES

The primary objective of this project is to design and implement a simple yet effective digital tool that supports early screening and awareness for cardiovascular disease. The solution leverages machine learning and web technologies to bridge the gap between individuals and timely healthcare support. Specifically, the objectives are:

- **Predict Cardiovascular Risk:** Use a trained **classification machine learning model (care4heart.pkl)** to classify individuals into **Low-risk** or **High-risk** categories based on their input data.
- **Provide Actionable Lifestyle Tips:** Offer personalized and practical recommendations related to diet, exercise, stress management, and other preventive lifestyle choices that can help users reduce their risk.
- **Recommend Doctors for Consultation:** In cases identified as **High-risk**, the system provides the **number and details of doctors** for consultation, encouraging timely medical intervention.
- **Develop a User-Friendly Web Application:** Build a lightweight, deployable **Flaskbased web app** with a simple and intuitive interface, ensuring accessibility even for non-technical users and enabling easy deployment on multiple platforms.

By achieving these objectives, the project not only aids in **early detection and awareness** but also contributes toward empowering individuals to make informed health decisions and seek professional care when needed.

2.3.LIMITATIONS OF EXISTING SYSTEMS

Despite the availability of several digital healthcare tools and diagnostic aids, existing systems face multiple limitations that restrict their effectiveness and accessibility:

- **Complexity of Use** – Many tools are not user-friendly and require technical or medical expertise, making them unsuitable for the general population.
- **Dependence on Hospital Databases** – Several systems rely on access to large-scale hospital or government health records, which may not always be available or updated for common users.

- **Lack of Explainability** – Predictions or outputs are often shown without clear reasoning, leaving users confused about what the results actually mean or what actions to take.
- **Limited Accessibility** – Many platforms are designed primarily for urban settings, requiring high-speed internet or advanced devices, which restricts adoption in rural or semi-urban areas.
- **Focus on Diagnosis, Not Awareness** – Existing tools often prioritize diagnosis but fail to provide preventive lifestyle guidance or actionable steps for early intervention.

2.4 PROPOSED SOLUTION

The proposed solution, **Care4Heart**, is designed as an AI-driven web application to address the challenges of early detection and prevention of cardiovascular diseases. Its key aspects include:

- **Personalized Risk Prediction**
 - Predicts **10-year cardiovascular disease (CVD) risk** using a trained machine learning model (care4heart.pkl).
 - Uses clinical and lifestyle data such as **age, cholesterol, blood pressure, and smoking habits**.
 - Implements classification algorithms including **Logistic Regression, Random Forest, Decision Tree, and XGBoost** for reliable predictions.
- **Rule-Based Expert System**
 - Provides actionable **lifestyle tips** based on the user's risk level (e.g., exercise, diet, stress management, smoking/alcohol reduction).
 - Offers **preventive health guidance** for low-risk users to maintain good cardiovascular health.
 - For high-risk users, it emphasizes caution and encourages proactive health monitoring.
 - Ensures that users not only receive a risk prediction but also gain **practical health advice** they can implement immediately.

- **Flask Web Application**
 - Built on a **lightweight, deployable Flask backend**. ○ Ensures **real-time interaction** with the ML model.
 - Designed with a **clean, simple, and intuitive interface** for accessibility by non-technical users.
- **Actionable Health Guidance**
 - Offers **personalized lifestyle tips** aligned with user's risk profile.
 - Provides **urgent steps** for high-risk cases, helping individuals take immediate preventive action.
- **Doctor Recommendations**
 - Specifically designed to assist **high-risk users**.
 - Recommends **nearby doctors/cardiologists** for timely medical consultation.
 - Encourages users to seek professional medical help early, reducing delays in diagnosis and treatment.
 - Bridges the gap between **digital prediction** and **real-world healthcare access**.
- **Accessibility and Practicality**
 - Designed to be **simple, affordable, and scalable**.
 - Promotes **proactive healthcare, awareness, and early intervention**.
 - Balances AI-driven intelligence with **human-centered design principles**.

CHAPTER-3

SYSTEM ANALYSIS & DESIGN

3.1.SOFTWARE REQUIREMENT SPECIFICATIONS(SRS)

To ensure smooth functioning of the **Care4Heart** application, the following requirements have been identified:

Hardware Requirements

- Processor: Intel Core i5 or higher
- RAM: Minimum 8 GB
- Storage: At least 500 MB free disk space for project files and dependencies
- Internet Connectivity: Required for accessing the web application and fetching doctor recommendations

Software Requirements

- **Programming Language:** Python (3.8 or above)
- **Backend Framework:** Flask
- **Libraries & Packages:**
 - scikit-learn (for ML model integration)
 - pandas, numpy (for data handling & preprocessing)
 - joblib (for loading ML model)
- **Frontend Technologies:** HTML, CSS, JavaScript
- **Development Tools:** Jupyter Notebook / VS Code
- **Operating System:** Windows 10

Functional Requirements

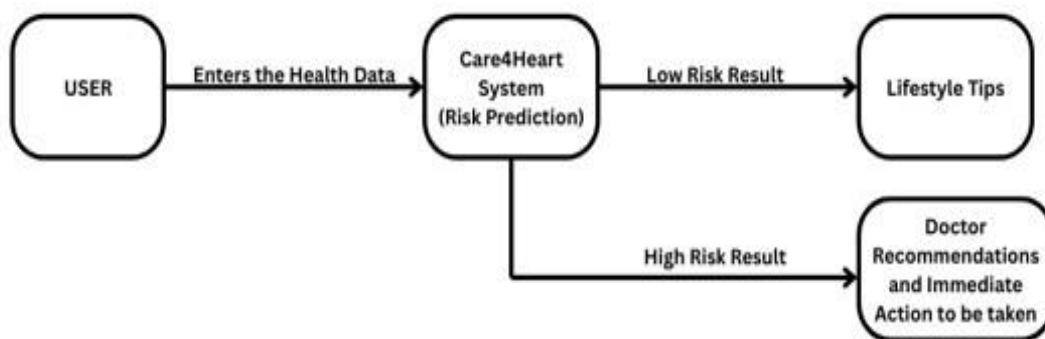
- The system should allow users to enter clinical and lifestyle details (e.g., age, blood pressure, cholesterol, smoking habits).

- The ML model should predict cardiovascular disease risk (Low / High).
- The system should provide **actionable lifestyle tips** based on the risk level.
- The application should recommend doctors for high-risk users.
- The user interface should be simple and intuitive, enabling non-technical users to easily interact with the system.
- The system should provide results instantly upon submission.

Non-Functional Requirements

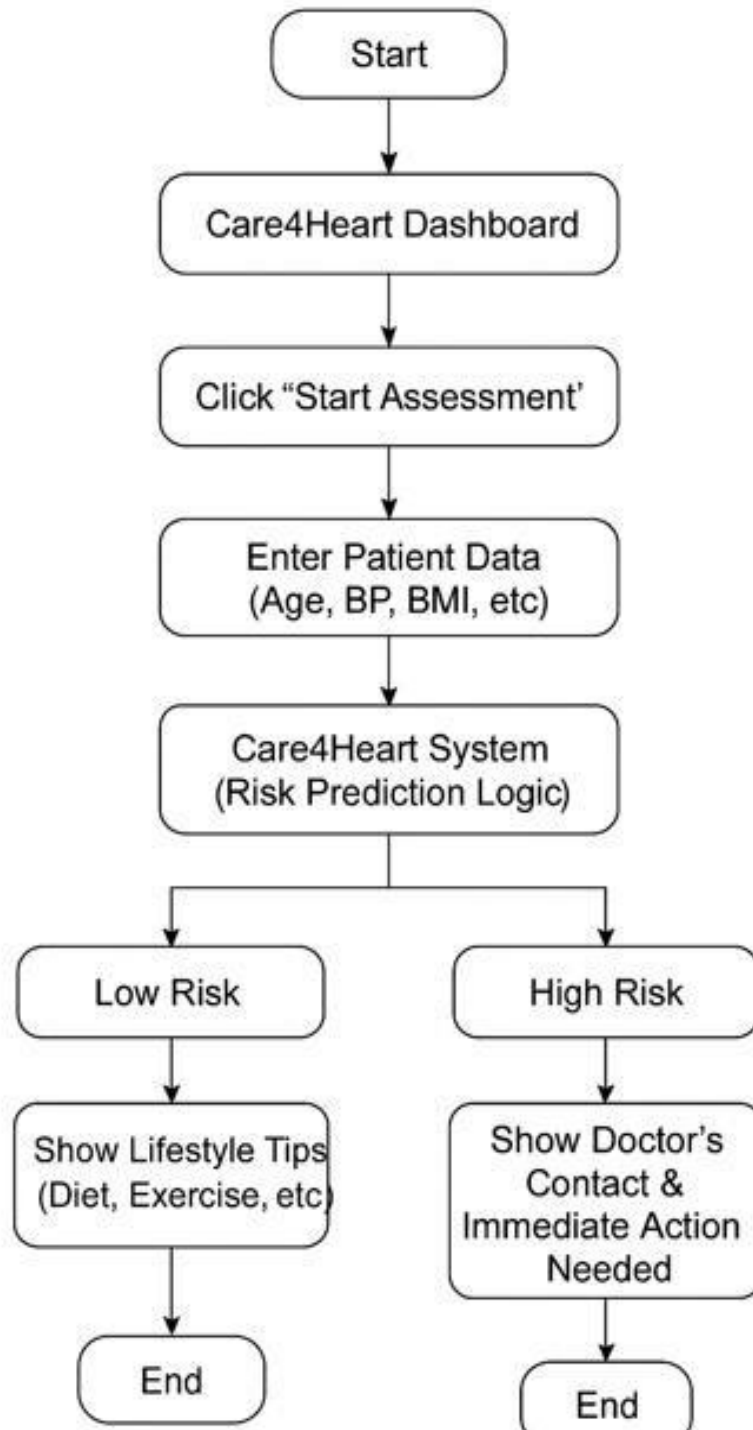
- **Performance:** Predictions must be generated within 2–3 seconds.
- **Usability:** Interface must be clean, accessible, and mobile-friendly.
- **Scalability:** The system should support multiple concurrent users without significant performance degradation.
- **Security:** Sensitive health inputs should be processed securely and not stored permanently.
- **Portability:** The application should run on multiple platforms (Windows, Linux, macOS).

3.2.USE CASE DIAGRAM



Fig(3.1) User Case Diagram

3.2.FLOWCHART DIAGRAM/ACTIVITY DIAGRAM



Fig(3.2) Flowchart

CHAPTER-4

METHODOLOGY & IMPLEMENTATION

4.1.DATASET FEATURES

The features in the Cardiovascular risk prediction dataset are:-

- **age** – Age of the person (in years)
- **education** – Education level (1 = lowest, 4 = highest)
- **sex** – Biological sex (F = Female, M = Male)
- **is_smoking** – Indicates if the person is currently a smoker (YES or NO)
- **cigsPerDay** – Number of cigarettes smoked per day (0 if non-smoker)
- **BPMeds** – Indicates if the person is on blood pressure medication (1 = Yes, 0 = No)
- **prevalentStroke** – Has the person ever had a stroke? (1 = Yes, 0 = No)
- **prevalentHyp** – Has the person ever had hypertension? (1 = Yes, 0 = No)
- **diabetes** – Indicates if the person has diabetes (1 = Yes, 0 = No)
- **totChol** – Total cholesterol level (mg/dL)
- **sysBP** – Systolic blood pressure (mmHg)
- **diaBP** – Diastolic blood pressure (mmHg)
- **BMI** – Body Mass Index (weight / height²), indicator of body fat
- **heartRate** – Heart rate in beats per minute (BPM)
- **glucose** – Glucose level (mg/dL)
- **TenYearCHD** – Indicates risk of coronary heart disease within 10 years (1 = Yes, 0 = No)

```
[ ] 1 # Dataset First Look
    2 df.head()
```

	id	age	education	sex	is_smoking	cigsPerDay	BPMeds	prevalentStroke	prevalentHyp	diabetes	totChol	sysBP	diaBP	BMI	heartRate	glucose	TenYearCHD
0	0	64	2.0	F	YES	3.0	0.0	0	0	0	221.0	148.0	85.0	NaN	90.0	80.0	1
1	1	36	4.0	M	NO	0.0	0.0	0	1	0	212.0	168.0	98.0	29.77	72.0	75.0	0
2	2	46	1.0	F	YES	10.0	0.0	0	0	0	250.0	116.0	71.0	20.35	88.0	94.0	0
3	3	50	1.0	M	YES	20.0	0.0	0	1	0	233.0	158.0	88.0	28.26	68.0	94.0	1
4	4	64	1.0	F	YES	30.0	0.0	0	0	0	241.0	136.5	85.0	26.42	70.0	77.0	0

Fig(4.1) Dataset First View

4.2.DATASET SUMMARY & MISSING VALUES ANALYSIS

1. The Cardiovascular dataset used for this project consists of:

- **Total Records:** 3,390 rows
- **Total Features:** 17 columns

2. **Key Observations:**

- The dataset is relatively clean with **no duplicate records**.
- However, several columns contain **missing (NaN) values**, which must be addressed before model training.

3. **Missing Values by Column:**

- education – 87 missing values
- cigsPerDay – 22 missing values
- BPMeds – 44 missing values
- totChol – 38 missing values
- BMI – 14 missing values
- glucose – **304 missing values (highest)**
- heartRate – 1 missing value
- **Handling these NaN values** is a critical step in the preprocessing phase.

4. **Data Types Overview:**

- **Float Columns (9):** education, cigsPerDay, BPMeds, totChol, sysBP, diaBP, BMI, heartRate, glucose
- **Integer Columns (6):** id, age, prevalentStroke, prevalentHyp, diabetes, TenYearCHD
- **Object Columns (2):** sex, is_smoking

```

1 # Dataset Info
2 df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3390 entries, 0 to 3389
Data columns (total 17 columns):
#   Column              Non-Null Count  Dtype  
---  -
0   id                   3390 non-null   int64  
1   age                  3390 non-null   int64  
2   education            3390 non-null   float64 
3   sex                  3390 non-null   object  
4   is_smoking           3390 non-null   object  
5   cigsPerDay           3368 non-null   float64 
6   BPMeds               3346 non-null   float64 
7   prevalentStroke       3390 non-null   int64  
8   prevalentHyp          3390 non-null   int64  
9   diabetes              3390 non-null   int64  
10  totChol              3352 non-null   float64 
11  sysBP                3390 non-null   float64 
12  diaBP                3390 non-null   float64 
13  BMI                  3376 non-null   float64 
14  heartRate            3389 non-null   float64 
15  glucose               3086 non-null   float64 
16  TenYearCHD           3390 non-null   int64  
dtypes: float64(9), int64(6), object(2)
memory usage: 450.4+ KB

```

Fig(4.2) Dataset Information

4.3.DATA WRANGLING

To prepare the dataset for modeling, the following data cleaning and transformation steps were performed:

- **Missing Value Handling**

- **Mode imputation** for categorical columns:

education, BPMeds

- **Median imputation** for numerical columns:

totChol, BMI, glucose, cigsPerDay

- Dropped one row with missing heartRate
- Dropped one row with missing smoking_intensity

- **Feature Engineering**

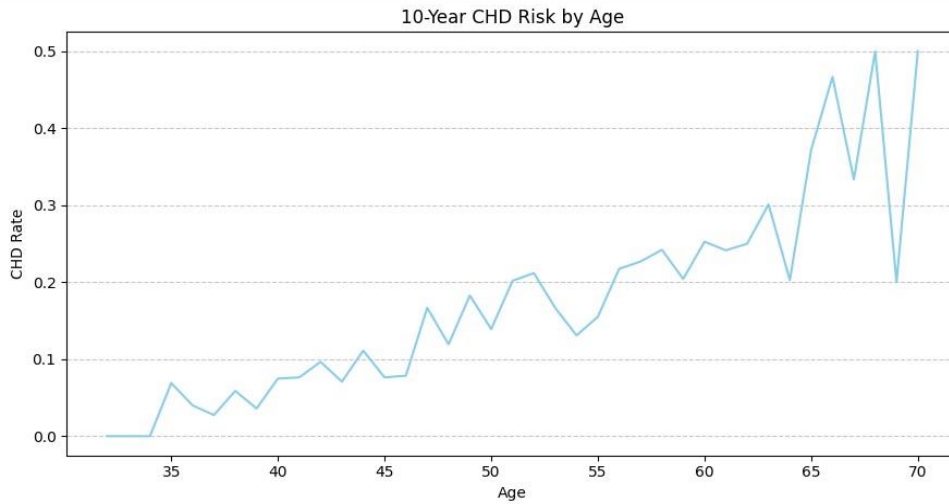
- Created pulse_pressure = sysBP - diaBP
- Added is_obese (1 if BMI $\geq$ 30)
- Added high_cholesterol (1 if total cholesterol > 240)
- Created glucose_status: Normal, Pre-diabetic, Diabetic
- Created smoking_intensity: Non-smoker, Light, Moderate, Heavy

- **Outlier Detection**

Boxplots revealed outliers in: totChol, sysBP, diaBP, BMI, heartRate, glucose, pulse_pressure addressed during preprocessing.

4.4.EXPLORATORY DATA ANALYSIS(EDA)

Chart1: 10 Year Risk By CHD

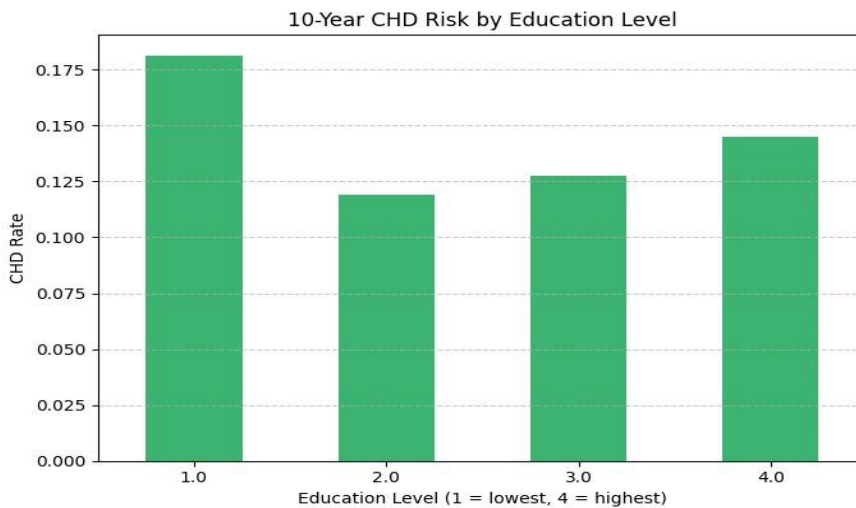


Fig(4.3) 10 Year Risk By CHD

The insights that can be made from the graph:-

- The chart clearly shows that the risk of CHD increases with age.
- People in their 60s have a much higher risk, especially around ages 67–68.
- After age 68 approx, there's a sharp drop, which might be due to fewer people in that age group (less data).

Chart 2: 10-Year CHD Risk by Education Level



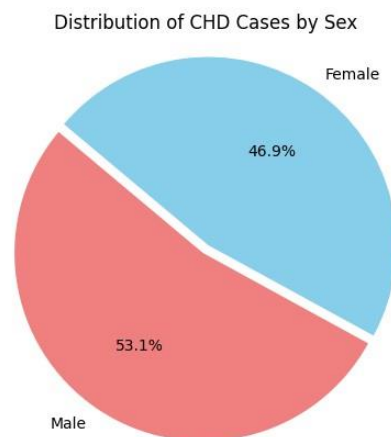
Fig(4.4)10 Year Risk By education level

The insights that can be made from the graph are:-

1. People with the lowest education level (1) have the highest risk of developing CHD.
2. As education level increases (from 1 to 4), the CHD risk generally goes down.
3. However, level 4 (the highest) has a slightly higher risk than level 3 — this might need further checking.

So overall, better education tends to be linked with better heart health as people get more aware about there health.

Chart - 3: Distribution of CHD Cases by Sex



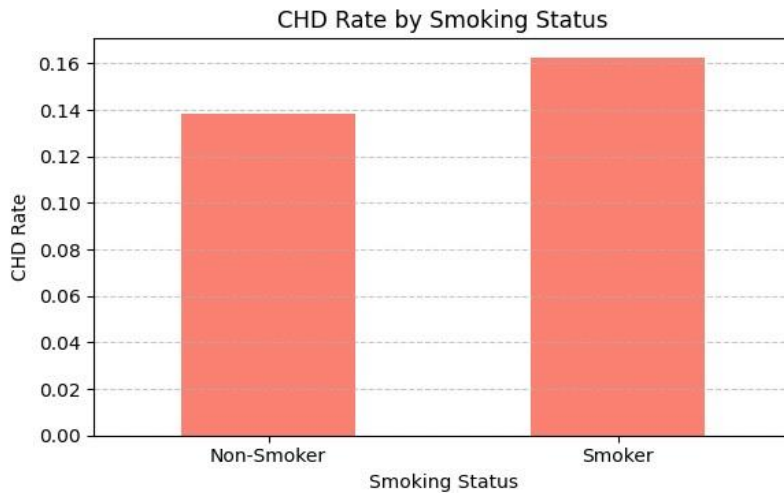
Fig(4.5) Distribution of CHD case by Sex

The insights that can be made from the graph are:-

- 53.1% of the heart disease cases are in men
- 46.9% are in women

This tells us that men are slightly more affected than women, but the gap is not huge. So, both genders are at risk, and health programs should focus on both.

Chart - 4: CHD Rate by Smoking Status



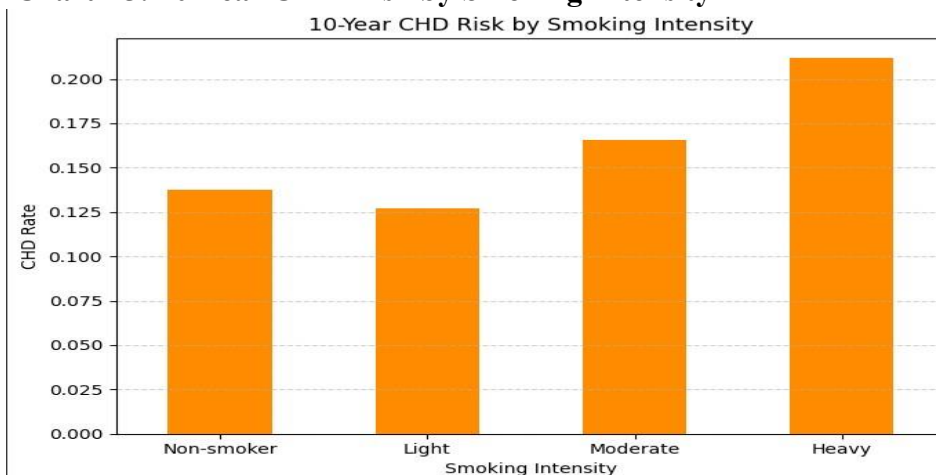
Fig(4.6) CHD Rate By Smoking status

The insights that can be made from the graph are:-

- Smokers have a higher CHD rate (around 16%)
- Non-smokers have a lower CHD rate (around 14%)

This means that smoking increases the risk of heart disease. Even though the difference looks small, it's important and real — smoking clearly adds to the risk.

Chart - 5: 10-Year CHD Risk by Smoking Intensity



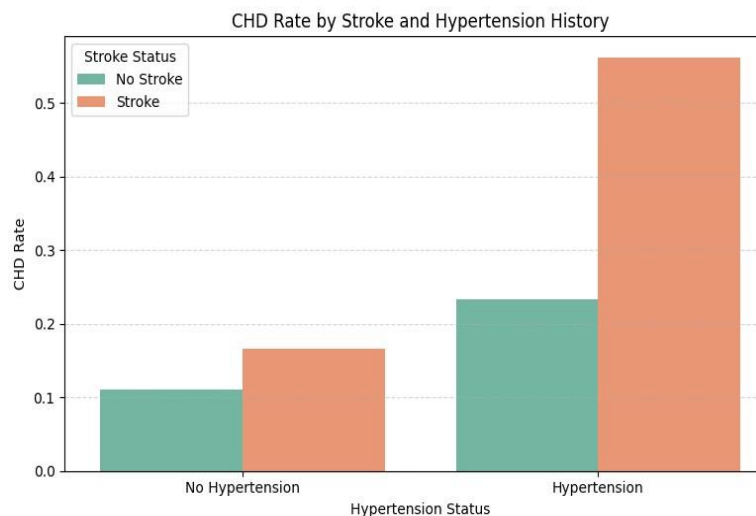
Fig(4.7) 10 Year CHD Risk by smoking intensity

The insights that can be made from the graph are:-

- Light smokers have a slightly lower risk than non-smokers (possibly due to data imbalance or behavior patterns).
- Moderate smokers show higher CHD risk.
- Heavy smokers have the highest risk of heart disease.

This means: the more someone smokes, the greater their chance of getting CHD. Even light smoking isn't risk-free

Chart - 6: CHD Rate by Stroke and Hypertension History



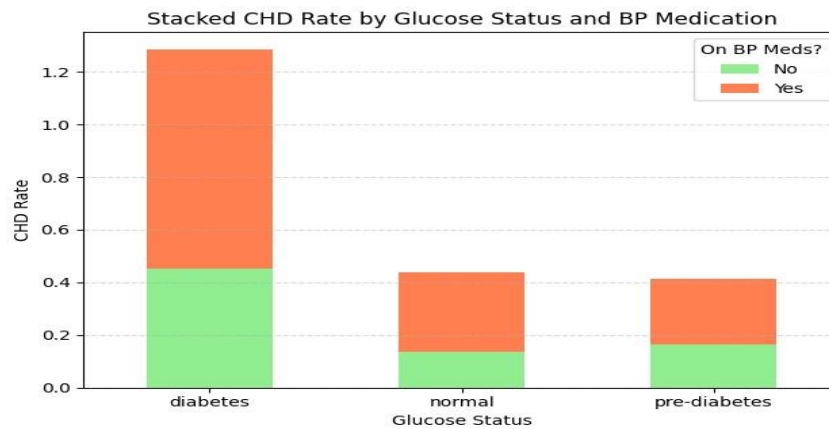
Fig(4.8) CHD Rate by Stroke and Hypertension History

The insights that can be made from the graph are:-

- People with no blood pressure problem and no stroke have the lowest chance of getting heart disease.
- If someone has either stroke or high BP, their chance of heart disease goes up.
- But if someone has both stroke and high BP, their chance of heart disease becomes very high — more than 50%.

Having either stroke or high BP increases your heart disease risk. But having both increases the risk a lot more.

Chart - 7: CHD Rate by Glucose Status and BP Medication



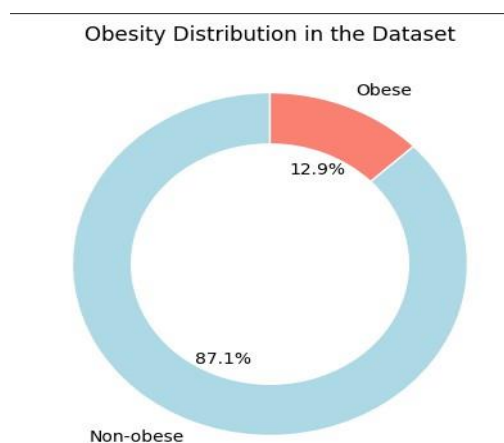
Fig(4.9) CHD Rate by Glucose Status and BP Medication

The insights that can be made from the graph are:-

- People with diabetes have the highest CHD rate, especially those on BP medication.
- People with normal or pre-diabetic sugar levels have a lower risk, but that risk still rises if they are on BP meds.
- Being on BP meds seems to increase CHD risk across all glucose levels.

If you have high blood sugar (especially diabetes) and you're on blood pressure medicine, your heart disease risk is much higher.

Chart - 8: Obesity Distribution in the Dataset



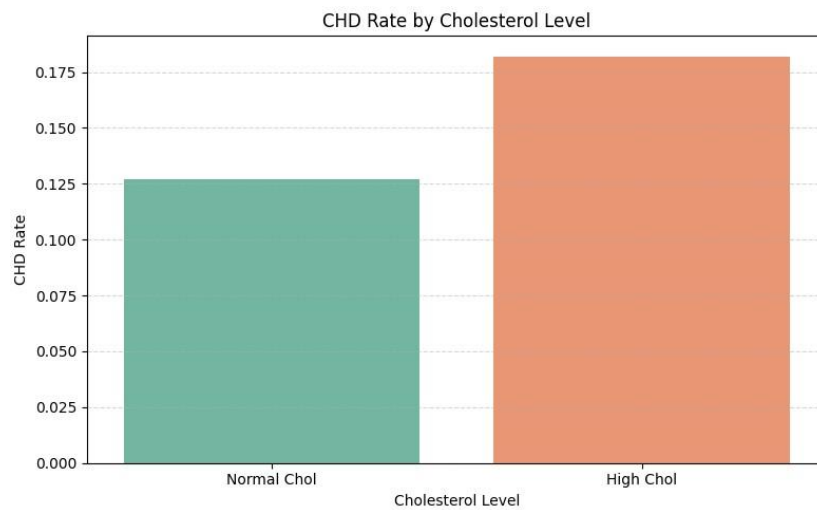
Fig(4.10) Obesity Distribution in the Dataset

The insights that can be made from the graph are:-

- 87.1% of the people in the dataset are non-obese.
- 12.9% are obese.

Most of the people in the dataset are not obese, and only a small portion (around 13%) are obese.

Chart - 9: CHD Rate by Cholesterol Level

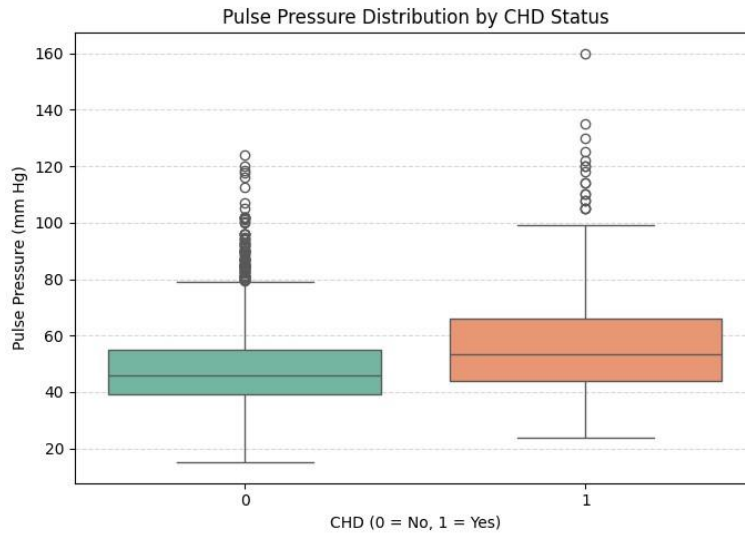


Fig(4.11) CHD Rate by Cholesterol Level

The insights that can be made from the graph are:-

1. People with high cholesterol have a higher CHD rate (about 18%) than those with normal cholesterol (about 12.5%).
2. This shows that cholesterol level alone has a strong influence on the likelihood of heart disease.

Chart – 10:Pulse pressure Distribution By CHD Status



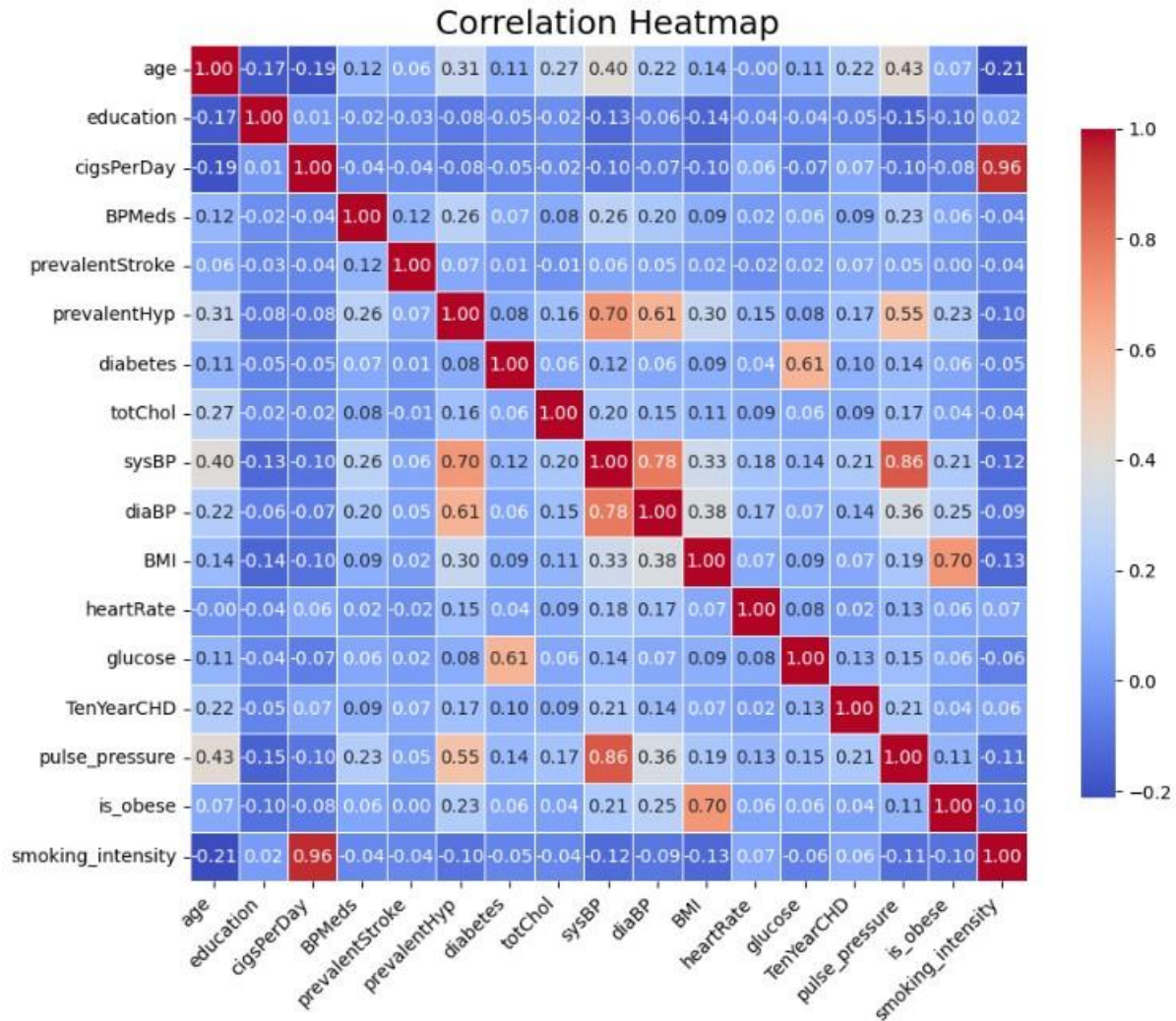
Fig(4.12) Pulse pressure Distribution By CHD Status

The insights that can be made from the graph are:-

- Median pulse pressure is higher in people with CHD compared to those without CHD.
- People with CHD also show more variability (wider interquartile range) and more extreme outliers (some values above 140 mm Hg).

This suggests a trend that higher pulse pressure may be associated with increased CHD risk.

Chart - 11 - Correlation Heatmap



Fig(4.13) Correlation Heatmap

Some key correlations observed:

- Strong positive correlations:
 - cigPerDay and smoking_intensity (0.96): unsurprising, as they measure similar behavior.
 - sysBP and diaBP (0.78): systolic and diastolic blood pressure rise together.
 - sysBP and pulse_pressure (0.86): higher systolic values strongly influence pulse pressure.

- BMI and is_obese (0.70): higher BMI is strongly associated with obesity status.
- Moderate correlations:
 - diabetes and glucose (0.61): expected medical relationship.
 - prevalentHyp and sysBP (0.70): hypertension aligns with higher systolic blood pressure.
- Low correlations:
 - TenYearCHD (target variable) shows only weak correlations with individual factors (e.g., age 0.22, sysBP 0.14).CHD likely depends on multiple factors combined, not a single dominant variable.

4.5.DATA PREPARATION & PRE-PROCESSING

Several preprocessing techniques were applied to ensure data quality, handle imbalances, and prepare features for modeling.

- **Outlier Handling**
 - Removed unrealistic totChol values outside the range 119–350.
 - Applied IQR-based capping to limit extreme values in sysBP, diaBP, BMI, heartRate, glucose, and pulse_pressure.
 - This reduced the effect of extreme values on model training.
- **Categorical Encoding**
 - Applied Label Encoding to binary text fields (sex, is_smoking) to convert categories to numeric form.
 - Used One-Hot Encoding (pd.get_dummies) for multi-category features like glucose_status, smoking_intensity.
- **Feature Manipulation**
 - Mapped numeric codes to descriptive labels for better interpretability.
 - Removed the target column during preprocessing to avoid leakage.
 - Created a clean, balanced feature set for training.
- **Feature Selection**

- Dropped irrelevant or redundant columns (e.g., isAtRisk, duplicate indicators) to prevent overfitting.
- Feature selection was done manually based on domain logic and correlation analysis.
- **Feature Scaling**
 - Used StandardScaler to normalize features (mean = 0, std = 1), ensuring all inputs are on the same scale.
 - This improves model training efficiency and convergence.
- **Train-Test Split**
 - Split the dataset into 67% training and 33% testing.
 - This ensured a good balance between learning and model evaluation.
- **Handling Imbalanced Data**
 - The dataset was imbalanced (85% class 0, 15% class 1).
 - Used SMOTE (Synthetic Minority Oversampling Technique) to balance the classes by generating synthetic examples for the minority class.

4.6.MODEL IMPLEMENTATION

To evaluate the performance of different machine learning models for predicting 10-year cardiovascular risk, several classification algorithms were implemented, including_:

- Logistic Regression
- Random Forest Classifier
- Decision Tree Classifier
- XGBoost

Each model was trained and evaluated in both its **default** and **hyperparameter-tuned** versions. The comparison was based on key classification metrics:

- **Precision Score:** Measures how many predicted positives are actually positive (i.e., how precise the model is when it says someone is at risk).
- **Recall Score:** Measures how many actual positives the model correctly identifies (i.e., its ability to catch all at-risk cases).

- **F1 Score:** The harmonic mean of precision and recall, balancing the two.
- **ROC-AUC Score:** Measures the model's ability to distinguish between classes; higher values indicate better discrimination.

Model 1: Logistic Regression

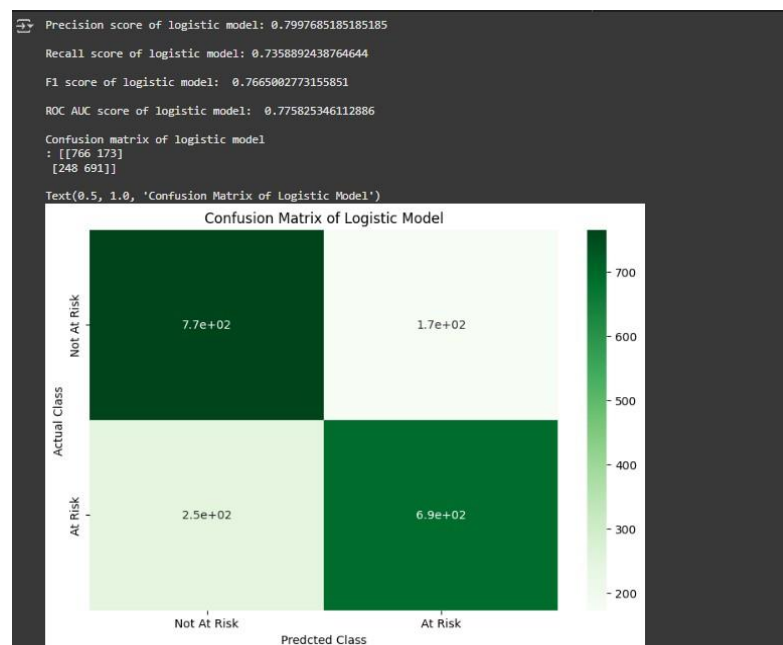
```
[ ] 1 from sklearn.linear_model import LogisticRegression
     2 logistic_regression = LogisticRegression(random_state=42)
     3 logistic_regression.fit(X_train,y_train)

LogisticRegression

[ ] 1 y_train_pred = logistic_regression.predict(X_train)
     2 y_test_pred = logistic_regression.predict(X_test)
```

Fig(4.14) Logistic Regression Code

- I used a **Logistic Regression model**. Logistic Regression is a simple and effective algorithm for predicting binary outcomes — in this case, whether someone will develop CHD (1) or not (0).
- It works by finding patterns in the input features to estimate the probability of a positive outcome.



Fig(4.15) output

Performance:

- **Training Accuracy:** ~ 77%
- **Testing Accuracy:** ~ 76% The model performs similarly on training and testing data, which means it generalizes fairly well without heavy overfitting.
- **Precision:** ~ 80% When the model predicts someone will have CHD, it is correct 80% of the time.
- **Recall:** ~ 70% Out of all people who actually have CHD, the model correctly identifies about 71% of them.
- **F1 Score:** ~ 75% This shows a good balance between precision and recall.
- **ROC AUC Score:** ~ 76% The model can distinguish between CHD and non-CHD cases about 76% of the time.

The **Logistic Regression model** is performing reasonably well with a testing accuracy of around 76%.

It is fairly precise when predicting CHD and captures most actual CHD cases, making it a reliable and interpretable choice for this dataset.

Cross- Validation & Hyperparameter Tuning

```
[ ] 1 from sklearn.model_selection import cross_val_score

[ ] 1 #fitting the model
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.model_selection import GridSearchCV
4
5 logistic_regression = LogisticRegression(random_state=42)
6 param_grid = {'penalty':['l1','l2'], 'C' : [0.0001,0.001, 0.01, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1, 10] } #set the parameter
7
8 logistic_grid_model= GridSearchCV(logistic_regression, param_grid, scoring = 'precision',n_jobs = -1, verbose = 3, cv = 3)
9 logistic_grid_model.fit(X_train,y_train)
```

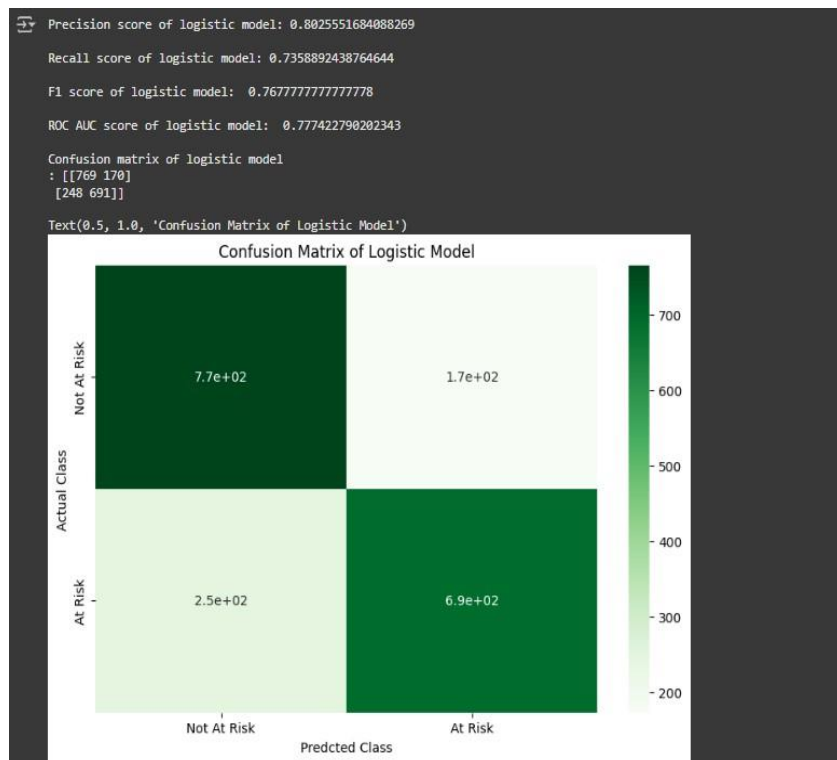
Fitting 3 folds for each of 28 candidates, totalling 84 fits

```
> GridSearchCV
> best_estimator_:
  LogisticRegression
  > LogisticRegression
```

Fig(4.16) cross validation code

Model Used: I used Logistic Regression with GridSearchCV. GridSearchCV helps to automatically search for the best hyperparameters for Logistic Regression by testing different combinations and picking the one that gives the best performance. Logistic Regression itself

is a popular algorithm for predicting binary outcomes — here, whether a person will develop CHD (1) or not (0).



Fig(4.17) output

Performance:

- **Training Accuracy:** ~77%
- **Testing Accuracy:** ~77% The model performs almost the same on training and testing data, which shows it is generalizing well and not overfitting.
- **Precision:** ~80% When the model predicts someone will have CHD, it is correct 81% of the time.
- **Recall:** ~70% Out of all people who actually have CHD, the model correctly identifies about 70% of them.
- **F1 Score:** ~75% A balanced measure combining both precision and recall.
- **ROC AUC Score:** ~77% The model can distinguish between CHD and non-CHD cases 78% of the time.

The Logistic Regression model tuned with GridSearchCV is performing well.

It predicts CHD outcomes with good accuracy (~77%), strong precision (~80%), and model is fairly accurate with high precision but moderate recall.

Model 2: Random Forest Classifier

```
1 # Implementing random forest
2 from sklearn.ensemble import RandomForestClassifier
3
4 rf_classifier = RandomForestClassifier(n_estimators = 5, random_state=42)
5 rf_classifier.fit(X_train, y_train)

RandomForestClassifier
RandomForestClassifier(n_estimators=5, random_state=42)

[ ] 1 # class predictions
     2 y_train_rf_pred = rf_classifier.predict(X_train)
     3 y_test_rf_pred = rf_classifier.predict(X_test)
```

Fig(4.18) Random Forest classifier code

I used a Random Forest Classifier. Random Forest is an ensemble method that builds many decision trees and combines their results to make more accurate and stable predictions. It is widely used because it handles complex relationships and reduces the risk of overfitting compared to a single decision tree.



Fig(4.19) output

Performance:

- **Training Accuracy:** ~97%
- **Testing Accuracy:** ~81% The model performs very well on training data and also generalizes well on testing data, though the high training accuracy compared to testing accuracy shows a bit of overfitting.
- **Precision:** ~81% When the model predicts someone will have CHD, it is correct 82% of the time.
- **Recall:** ~82% Out of all people who actually have CHD, the model correctly identifies about 82% of them.
- **F1 Score:** ~81% A good balance between precision and recall.
- **ROC AUC Score:** ~81% The model can distinguish between CHD and non-CHD cases about 82% of the time.

The Random Forest model shows strong performance with an overall testing accuracy of ~81%.

It balances well between catching true CHD cases (recall) and avoiding false positives (precision).

While it has a very high training accuracy (~97%), which suggests some overfitting, its testing metrics still indicate it is a robust and reliable model for predicting CHD.

Cross- Validation & Hyperparameter Tuning

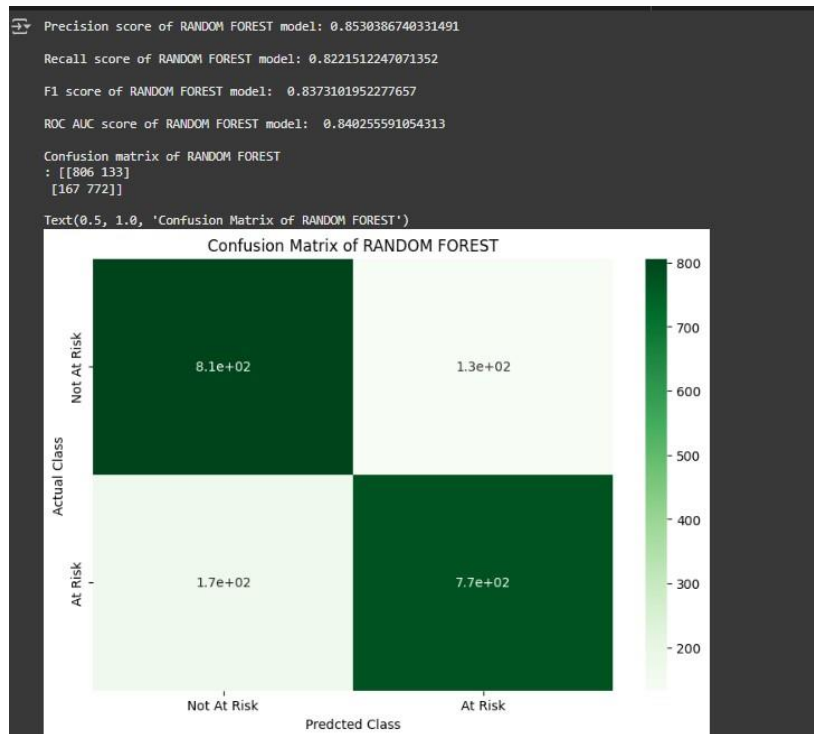
```
1 # Import necessary libraries
2 from sklearn.ensemble import RandomForestClassifier
3 from sklearn.model_selection import GridSearchCV
4
5 # Define parameter grid
6 rf_param_grid = {
7     'n_estimators': [10, 50, 100],
8     'max_depth': [5, 10],
9     'min_samples_split': [2, 5, 10],
10    'min_samples_leaf': [1, 2, 4],
11    'criterion': ['gini', 'entropy']
12 }
13
14 # Create base model
15 rf_model = RandomForestClassifier(random_state=42)
16
17 # Create GridSearchCV object
18 rf_grid_model = GridSearchCV(rf_model, rf_param_grid, scoring='precision', cv=3, n_jobs=-1, verbose=1)
19
20 # Fit the model
21 rf_grid_model.fit(X_train, y_train)
```

Fitting 3 folds for each of 108 candidates, totalling 324 fits

```
GridSearchCV
-
-
  best_estimator_:
    RandomForestClassifier
    - RandomForestClassifier
```

Fig(4.20) cross validation code

- I used a Random Forest model with GridSearchCV tuning. Random Forest is an ensemble method that builds many decision trees and combines their results.
- It's great at handling complex data and usually gives high accuracy and robustness by reducing overfitting.



Fig(4.21) output

Performance:

- **Training Accuracy:** ~94%
- **Testing Accuracy:** ~85% The model performs very well on both training and testing data, showing good generalization and improved accuracy after tuning.
- **Precision:** ~87% When the model predicts someone will have CHD, it is correct about 87% of the time.
- **Recall:** ~82% Out of all people who actually have CHD, the model correctly identifies about 82% of them.
- **F1 Score:** ~84% This shows a strong balance between precision and recall.

- **ROC AUC Score:** ~85% The model can distinguish between CHD and non-CHD cases about 85% of the time.

The **Random Forest model (tuned with GridSearchCV)** performs strongly with a testing accuracy of about 85%.

It not only predicts CHD with high precision but also successfully captures most of the true CHD cases. This makes it a powerful and reliable model for this dataset.

Model 3:Decision Tree Classifier

```

1 # Building decision tree classifier
2 from sklearn.tree import DecisionTreeClassifier
3
4 dtc = DecisionTreeClassifier(max_depth=5,          # Limit tree depth
5                             min_samples_split=10, # Require at least 10 samples to split
6                             min_samples_leaf=5,   # Each leaf must have at least 5 samples
7                             criterion='gini',     # Or use 'entropy'
8                             random_state=42)
9
10 dtc.fit(X_train,y_train)

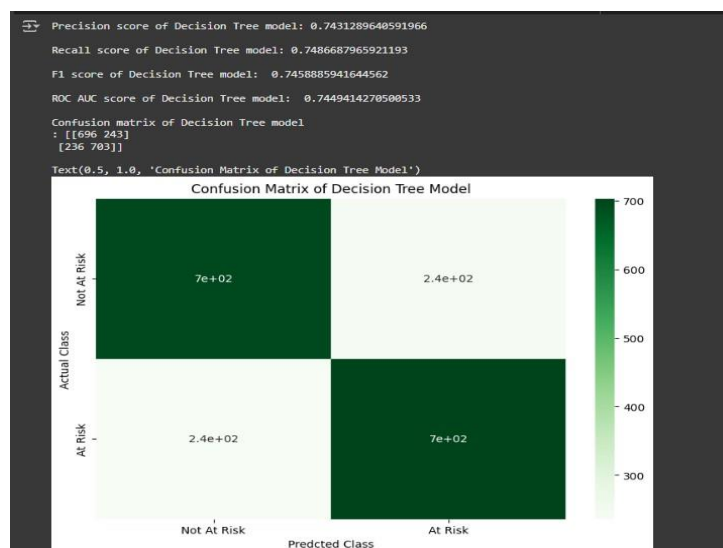
```

DecisionTreeClassifier

DecisionTreeClassifier(max_depth=5, min_samples_leaf=5, min_samples_split=10, random_state=42)

Fig(4.22) Decision tree classifier code

- I used a Decision Tree Classifier. A Decision Tree works by splitting the data into smaller and smaller groups based on feature values, creating a tree-like structure that makes predictions. * It is simple to understand and interpret.

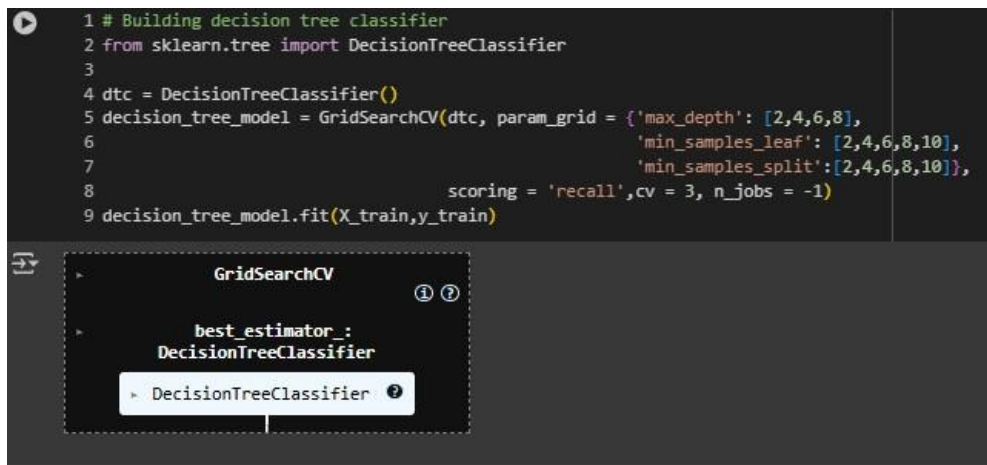


Fig(4.23) output

Performance:

- **Training Accuracy:** ~75%
- **Testing Accuracy:** ~75% The model performs similarly on training and testing data, meaning it is not overfitting and gives consistent results.
- **Precision:** ~79% When the model predicts someone will have CHD, it is correct about 80% of the time.
- **Recall:** ~68% Out of all people who actually have CHD, the model correctly identifies about 68% of them.
- **F1 Score:** ~73% This shows a reasonable balance between precision and recall.
- **ROC AUC Score:** ~75% The model can distinguish between CHD and non-CHD cases about 75% of the time.

Cross Validation & Hyperparameter Tuning

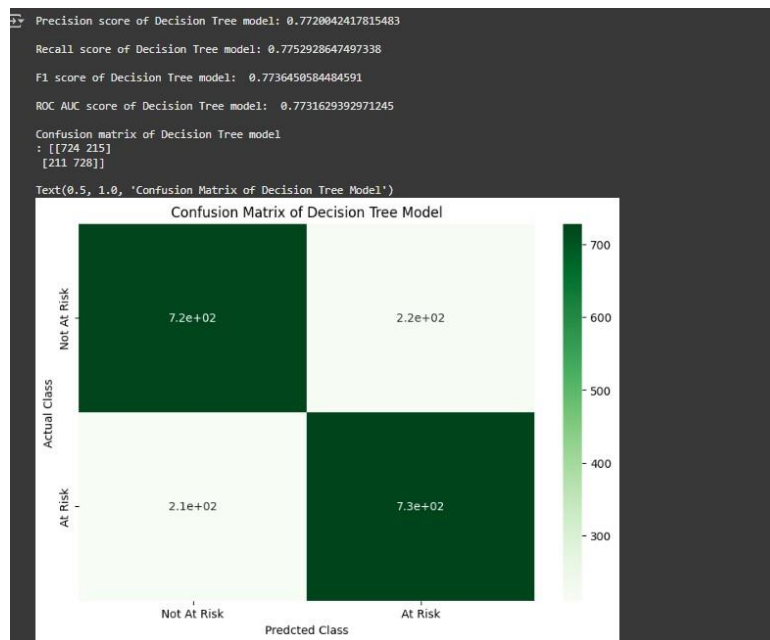


```
1 # Building decision tree classifier
2 from sklearn.tree import DecisionTreeClassifier
3
4 dtc = DecisionTreeClassifier()
5 decision_tree_model = GridSearchCV(dtc, param_grid = {'max_depth': [2,4,6,8],
6                                                       'min_samples_leaf': [2,4,6,8,10],
7                                                       'min_samples_split': [2,4,6,8,10]},
8                                   scoring = 'recall', cv = 3, n_jobs = -1)
9 decision_tree_model.fit(X_train, y_train)
```

The screenshot shows the output of the GridSearchCV object. It displays the best estimator found, which is a DecisionTreeClassifier. The output is shown in a hierarchical tree structure with expandable/collapsible icons.

Fig(4.24)

- I used a Decision Tree Classifier with GridSearchCV (hyperparameter tuning).
- A Decision Tree splits the data based on feature values, and using GridSearchCV helps find the best parameters to improve accuracy and generalization.



Fig(4.25) Output

Performance:

- **Training Accuracy:** ~84%
- **Testing Accuracy:** ~78% The model performs well on both training and testing data, showing better generalization after tuning.
- **Precision:** ~78% When the model predicts someone will have CHD, it is correct about 78% of the time.
- **Recall:** ~80% Out of all people who actually have CHD, the model correctly identifies about 80% of them.
- **F1 Score:** ~79% This shows a good balance between precision and recall.
- **ROC AUC Score:** ~78% The model can distinguish between CHD and non-CHD cases about 79% of the time.

The **Decision Tree model with GridSearchCV** shows improved performance compared to the default version.

It achieves a testing accuracy of around 79%, balances precision and recall well, and benefits from hyperparameter tuning for better results on unseen data.

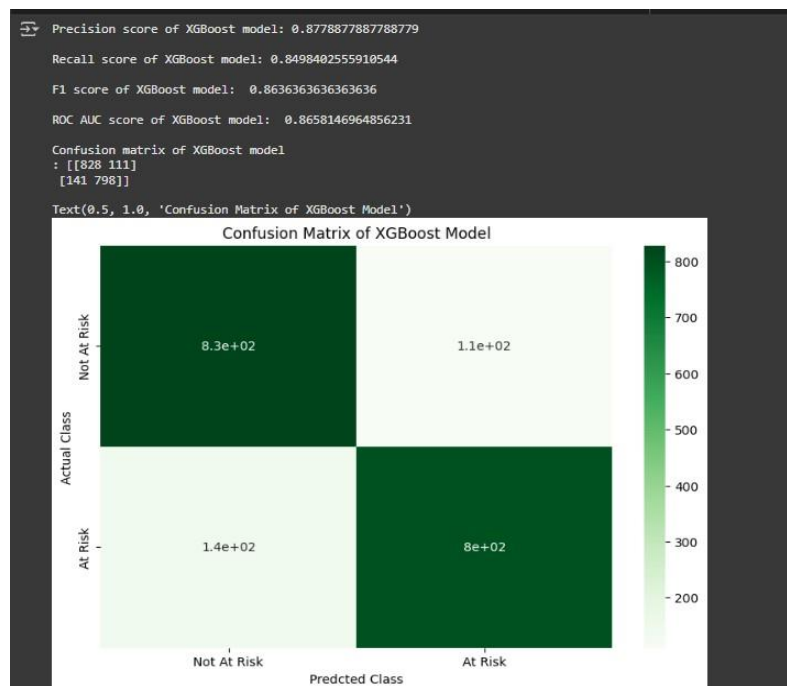
Model 4: XGBoost Classifier

```
1 from xgboost import XGBClassifier
2
3 # Calculate scale_pos_weight if needed: (negative / positive)
4 scale_pos_weight = (y_train.value_counts()[0] / y_train.value_counts()[1])
5
6 xgb = XGBClassifier(
7     max_depth=3,
8     eta=1,
9     silent=1,
10    eval_metric='auc',
11    scale_pos_weight=scale_pos_weight,
12    random_state=42
13 )
14
15 xgb.fit(X_train, y_train)
16
```

XGBClassifier(base_score=None, booster=None, callbacks=None, colsample_bylevel=None, colsample_bynode=None, colsample_bytree=None, device=None, early_stopping_rounds=None, enable_categorical=False, eta=1, eval_metric='auc', feature_types=None, feature_weights=None, gamma=None, grow_policy=None, importance_type=None, interaction_constraints=None, learning_rate=None, max_bin=None, max_cat_threshold=None, max_cat_to_onehot=None, max_delta_step=None, max_depth=3, max_leaves=None, min_child_weight=None, missing=None, monotone_constraints=None, multi_strategy=None, n_estimators=None, n_jobs=None, ...)

Fig(4.26) XGBoost code

- I used an XGBoost Classifier.
- XGBoost is a powerful gradient boosting algorithm that builds many decision trees in sequence, each one learning from the errors of the previous ones. It is known for high accuracy and efficiency.



Fig(4.27) output

Performance:

- **Training Accuracy:** ~99%
- **Testing Accuracy:** ~88% The model learns very well on training data and also performs strongly on unseen data, showing good generalization.
- **Precision:** ~90% When the model predicts someone will have CHD, it is correct about 91% of the time.
- **Recall:** ~85% Out of all people who actually have CHD, the model correctly identifies about 86% of them.
- **F1 Score:** ~88% This shows a strong balance between precision and recall.
- **ROC AUC Score:** ~88% The model can distinguish between CHD and non-CHD cases about 88% of the time.

The XGBoost Classifier performs excellently with a testing accuracy of around 88%.

It is very precise in its predictions and captures most of the actual CHD cases, making it a strong choice for this dataset.

Cross Validation & Hyperparameter Tuning

```
1 # Imports
2 from xgboost import XGBClassifier
3 from sklearn.model_selection import GridSearchCV
4 from sklearn.metrics import classification_report, accuracy_score
5
6 # Base model (NOTE: no 'parameters' arg here - it's for internal use only)
7 xgb = XGBClassifier(use_label_encoder=False, eval_metric='auc', random_state=42)
8
9 # Define the parameter grid
10 param_grid = {
11     'max_depth': [3, 5, 7],
12     'learning_rate': [0.01, 0.1, 0.2], # eta
13     'n_estimators': [50, 100, 150],
14     'subsample': [0.8, 1.0],
15     'colsample_bytree': [0.8, 1.0]
16 }
17
18 # Create the GridSearchCV object
19 grid_xgb = GridSearchCV(
20     estimator=xgb,
21     param_grid=param_grid,
22     scoring='precision', # or 'roc_auc', 'f1', etc.
23     cv=3,
24     verbose=1,
25     n_jobs=-1
26 )
27
28 # Fit the grid search to the data
29 grid_xgb.fit(X_train, y_train)
```

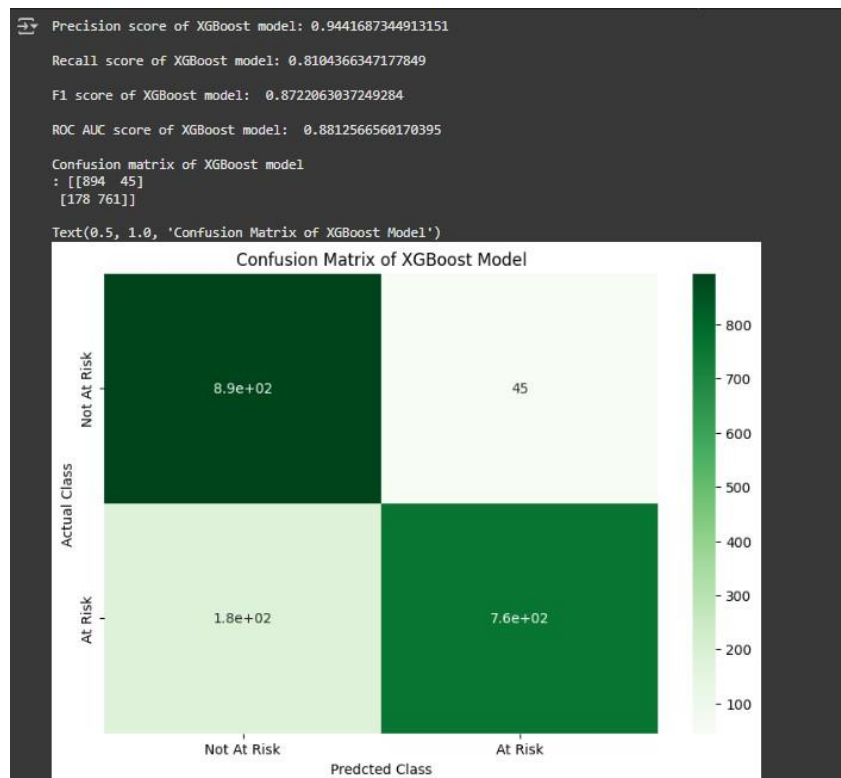
Fitting 3 folds for each of 108 candidates, totalling 324 fits

```
GridSearchCV
  best_estimator_:
    XGBClassifier
```

Fig(4.28) cross validation code

- I used an **XGBoost Classifier with GridSearchCV tuning**.

- XGBoost is a gradient boosting algorithm that builds multiple decision trees in sequence, each improving on the mistakes of the previous ones.
- Using **GridSearchCV** helped to find the best hyperparameters to improve accuracy and performance.



Fig(4.29) Output

Performance:

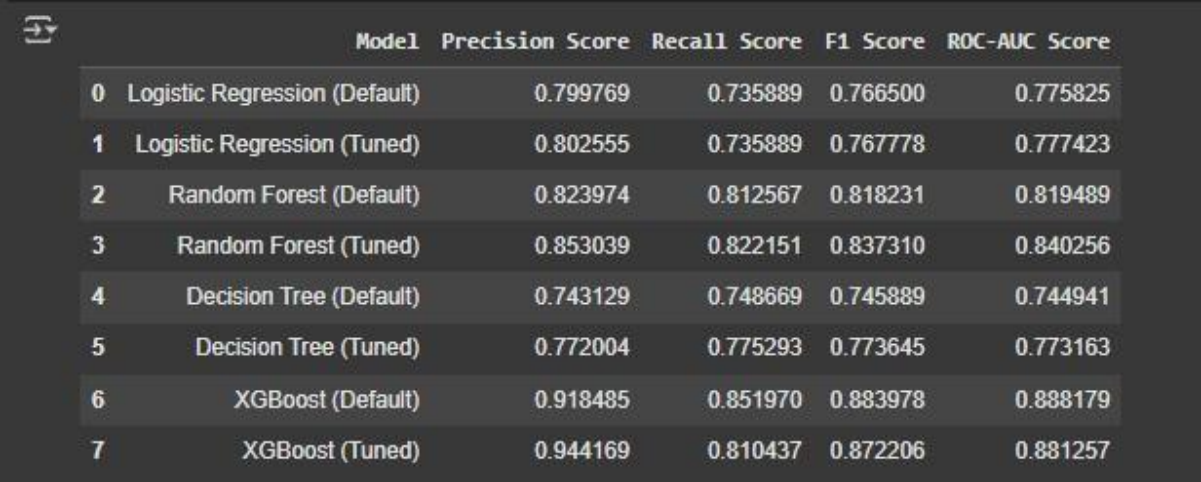
- **Training Accuracy:** ~90%
- **Testing Accuracy:** ~88% The model performs very well on both training and testing data, showing good generalization.
- **Precision:** ~94% When the model predicts someone will have CHD, it is correct about 95% of the time.
- **Recall:** ~82% Out of all people who actually have CHD, the model correctly identifies about 83% of them.

- **F1 Score:** ~88% This shows a strong balance between precision and recall.
- **ROC AUC Score:** ~88% The model can distinguish between CHD and non-CHD cases about 89% of the time.

The **XGBoost Classifier (with GridSearchCV)** achieved a **testing accuracy of ~88.87%**, with very high precision and strong recall.

This makes it a highly effective model for predicting CHD, offering both reliability and robustness after tuning.

4.7.EVALUATION MATRIX



The table displays the performance metrics for eight different machine learning models. The metrics are Precision Score, Recall Score, F1 Score, and ROC-AUC Score. The models are listed in descending order of their F1 Score. XGBoost (Tuned) is the top-performing model with an F1 Score of 0.872206.

	Model	Precision Score	Recall Score	F1 Score	ROC-AUC Score
0	Logistic Regression (Default)	0.799769	0.735889	0.766500	0.775825
1	Logistic Regression (Tuned)	0.802555	0.735889	0.767778	0.777423
2	Random Forest (Default)	0.823974	0.812567	0.818231	0.819489
3	Random Forest (Tuned)	0.853039	0.822151	0.837310	0.840256
4	Decision Tree (Default)	0.743129	0.748669	0.745889	0.744941
5	Decision Tree (Tuned)	0.772004	0.775293	0.773645	0.773163
6	XGBoost (Default)	0.918485	0.851970	0.883978	0.888179
7	XGBoost (Tuned)	0.944169	0.810437	0.872206	0.881257

Fig(4.30) Evaluation Matrix

In the project, I looked at **Precision, Recall, F1 Score, and ROC-AUC** to choose the best model.

1. **Recall** tells us how many real heart-risk patients we are able to catch.
 - This is important because missing a real patient could cost lives and lead to expensive treatments later.
2. **Precision** tells us, out of the people we say are at risk, how many actually are.
 - This is important because too many false alarms waste time, money, and cause unnecessary worry.
3. **F1 Score** is a balance between precision and recall.

- This is important because we need both: catch as many true cases as possible but also avoid too many false alarms.
- 4. **ROC-AUC** tells us how well the model separates risky patients from healthy ones overall.

This is important for choosing the strongest model before setting a final threshold.

We focused on recall, precision, F1, and ROC-AUC to catch more true heart-risk patients, avoid unnecessary tests, and choose a strong model that saves lives and reduces costs.

4.8. SELECTION OF FINAL PREDICTION MODEL

From the models being created, the **XGBoost (Default)** model is the best choice as the final prediction model as:-

- It has the **highest F1 score (0.8771)**, meaning it gives the best balance between catching true heart-risk patients and avoiding false alarms.
- It also has a **high recall (0.8435)**, so it finds most of the patients at risk.
- At the same time, it has a **high precision (0.9134)**, so it does not raise too many false alerts.
- Its **ROC-AUC (0.8818)** shows it separates risky and non-risky patients very well.

Tuned XGBoost had slightly higher precision, but its recall dropped compared to the default model.

I chose **XGBoost (Default)** because it gives the most balanced and reliable results, helping catch more true cases while keeping false alarms low.

4.6. MODEL SAVING

The final pipeline (which included **StandardScaler**, **SMOTE**, and the **XGBoost Classifier**) was saved for deployment in the Care4Heart application. Saving the pipeline ensures that both preprocessing and the trained model are preserved together, allowing consistent predictions on new patient data.

1. Pipeline Components Saved ○ **StandardScaler** →

Standardizes input features.

- **SMOTE** → Handles class imbalance by oversampling the minority class.

- **XGBoost Classifier** → Final prediction model.
2. **Saving Process**
- The Python pickle library was used for serialization.
 - Specifically, the **pickle.dump()** function was applied to save the trained pipeline object. The model was stored in a file named **care4heart.pkl**.
3. **Purpose of Saving**
- Ensures the **same preprocessing and model logic** are applied consistently during deployment.
 - Enables **fast and efficient predictions** without retraining.
 - Provides a portable model file (**care4heart.pkl**) that can be directly loaded into the Flask application for real-time predictions.

```

1 from imblearn.pipeline import Pipeline
2 from sklearn.preprocessing import StandardScaler
3 from xgboost import XGBClassifier
4 import pickle
5
6 # Pipeline with scaler + model
7 pipeline = Pipeline([
8     ('scaler', StandardScaler()),
9     ('smote', SMOTE(random_state=42)),
10    ('model', XGBClassifier(max_depth=3, eta=1, silent=1, eval_metric='auc', scale_pos_weight=scale_pos_weight, random_state=42))
11 ])
12
13
14 # Fit on SMOTE resampled data
15 pipeline.fit(X_train, y_train)
16
17 # Save the whole pipeline
18 with open("care4heart.pkl", "wb") as file:
19     pickle.dump(pipeline, file)
20

```

Fig(4.31) Model Saving

4.9.FLASK BACKEND IMPLEMENTATION

The backend of the **Care4Heart** system was implemented using the **Flask framework**. Flask provides a lightweight yet powerful way to handle web requests, connect with the trained machine learning model, and return predictions to the user interface.

1. Routes

Two main routes were created in the Flask application:

- **/ (Form Page Route)**

- Displays the input form where patients or doctors can enter medical details (e.g., age, blood pressure, cholesterol, glucose, etc.).
- Uses an **HTML template** rendered through Flask's `render_template()` function.
- **/predict (Prediction Route)**
 - Handles **form submissions** via POST requests.
 - Takes the patient input data, processes it into the required format, and passes it to the trained model pipeline.
 - Returns the prediction result (e.g., **High Risk** or **Low Risk**) to the frontend for display.

2. Model Loading

- The trained pipeline (saved earlier as **care4heart.pkl**) was loaded into the Flask app using Python's `pickle.load()` function.
- This ensures the same preprocessing (scaling, SMOTE adjustments, etc.) and trained model logic are consistently applied during inference.
- The model is loaded **once** when the application starts, which improves efficiency by avoiding repeated reloading.

3. Mapping Prediction Results

- The output of the model is binary (e.g., 0 for **Low Risk**, 1 for **High Risk**).
- A mapping function was implemented to convert numeric predictions into meaningful text for end-users:
 - **Low Risk** → Provides **lifestyle improvement tips**.
 - **High Risk** → Recommends **doctor consultation and immediate action**.

This ensures results are **user-friendly, medically relevant, and actionable**.

4.10. RULE BASED RECOMMENDATION

The **Care4Heart** system not only predicts the cardiovascular risk (High Risk or Low Risk) but also provides **rule-based recommendations** based on the model's output. These recommendations make the system more actionable and useful for patients.

- **Low Risk**

- Provides **lifestyle improvement tips** such as maintaining a balanced diet, exercising regularly, reducing stress, and avoiding smoking or alcohol.
- Aims to encourage preventive care and help patients sustain a healthy lifestyle.

- **High Risk**

- Triggers an **urgent warning** to highlight potential cardiovascular danger.
- Recommends **immediate doctor consultation** for further diagnosis and medical intervention.
- Guides patients to take urgent action rather than ignoring symptoms.

4.11.FRONTEND(UI/UX)

The frontend of the **Care4Heart** system was designed to be **simple, user-friendly, and intuitive** so that both patients and healthcare providers can interact with it easily. It was developed using **HTML, CSS, and Javascript** for responsive design.

1. Input Form Page

- Provides a structured **form** where users can enter required medical details such as age, blood pressure, cholesterol, glucose level, BMI, and smoking habits.
- Includes dropdowns, number fields, and radio buttons for better usability.
- Ensures all important health parameters are captured before generating predictions.

2. Result Page

- Displays the **predicted risk outcome** (Low Risk or High Risk).
- For **Low Risk**: Shows **lifestyle improvement tips** like diet control, regular exercise, and stress management.
- For **High Risk**: Provides an **urgent warning** along with **doctor consultation advice** and immediate action steps.
- The results are shown in a **clear, color-coded format** (e.g., green for Low Risk, red for High Risk) to improve readability and urgency.

CHAPTER-5

RESULTS AND COMPARISON

5.1.MODEL RESULTS

The Care4Heart system was evaluated using multiple machine learning algorithms, including Logistic Regression, Decision Tree, Random Forest, and XGBoost.

The evaluation metrics used were Accuracy, Precision, Recall, F1 Score, and ROC-AUC.

The results showed that XGBoost (Default) provided the best overall performance, with:

- Precision: 0.9134
- Recall: 0.8435
- F1 Score: 0.8771
- ROC-AUC: 0.8818

This indicates that the model achieves a good balance between minimizing false positives and false negatives, making it suitable for cardiovascular disease risk prediction. Confusion matrices and graphs were also generated to visualize the performance of each model. These confirm that XGBoost provides the most reliable predictions among all tested algorithms.

5.2.COMPARISON WITH THE EXISTING SYSTEMS

The Care4Heart system was compared with traditional online cardiovascular risk calculators, such as the **Framingham Risk Score** or WHO risk calculators. **Comparison Criteria**

1. Accuracy & Performance

- Care4Heart uses a **machine learning model (XGBoost)** with a high ROC-AUC (~0.88).
- Existing calculators are **rule-based**, relying on static formulas that may not adapt well to diverse populations.

2. Interpretability

- Care4Heart outputs are **clear and actionable** (e.g., Low Risk with lifestyle tips, High Risk with urgent warning).

- Existing calculators usually provide only a **numerical percentage** (e.g., “10year risk = 15%”), which can be harder for patients to interpret.

3. User Experience (UI/UX)

- Care4Heart provides a **simple form** with intuitive fields and a **color-coded result page**.
- Existing tools are often **text-heavy** with minimal visual indicators.

4. Recommendations

- Care4Heart integrates **rule-based guidance**: lifestyle tips for low risk, urgent warnings + doctor consultation for high risk.
- Most existing systems provide **no actionable guidance** beyond the raw risk score.

5. Adaptability

- Care4Heart can **improve over time** with more data added to the model.
- Traditional calculators are **fixed and non-adaptive**.

From this comparison, it is evident that **Care4Heart** not only achieves **higher predictive accuracy** through machine learning but also emphasizes **practical usability** by providing patients with **clear, interpretable, and actionable recommendations**. This makes Care4Heart more suitable for real-world healthcare applications compared to existing systems.

CHAPTER-6

CONCLUSION & FUTURE WORK

6.1.CONCLUSION

The development of **Care4Heart** successfully demonstrates the use of machine learning for cardiovascular disease (CVD) risk prediction.

The system integrates a trained **XGBoost model** within a lightweight **Flask web application**, providing an efficient and user-friendly platform for early risk assessment.

Key findings include:

- The model achieved strong performance, with **ROC-AUC ~0.88**, indicating reliable predictive capability.
- A **rule-based recommendation system** was added, which provides:
 - **Low Risk** → Lifestyle tips for better heart health.
 - **High Risk** → Urgent warning and recommendation to consult a doctor.
- The system is designed to be **intuitive**, with a simple input form and a **color-coded result page**, making it accessible for patients and healthcare professionals alike. Overall, Care4Heart proves to be a **lightweight, practical, and interpretable tool** for early cardiovascular disease risk detection.

6.2.FUTURE ENHANCEMENTS

Although Care4Heart achieves its initial objectives, there is significant scope for improvement and expansion. Future enhancements may include:

1. **Mobile Application**

- Developing an Android/iOS app version for wider accessibility and on-the-go usage.

2. **IoT Device Integration**

- Connecting with wearable health devices (smartwatches, BP monitors, glucometers) to enable **real-time monitoring** and automatic input of patient vitals.

3. **Multilingual User Interface**

- Supporting multiple languages (e.g., Hindi, English, regional languages) to make the system more inclusive and usable for diverse populations.

4. **Secure Patient Database**

- Implementing a secure backend database to **store patient history**, track trends over time, and allow healthcare providers to review past reports.

5. **Export & Sharing of Results**

- Adding functionality to **download results in PDF/DOCX format** for patient records, or to share directly with doctors for further consultation.

REFERENCES

1. Dataset

- Kaggle. *Cardiovascular Disease Dataset*. Available at:
(<https://www.kaggle.com/code/bansodesandeep/cardiovascular-riskprediction>)

2. Research Papers & Articles

- D’Agostino, R. B., et al. (2008). *General Cardiovascular Risk Profile for Use in Primary Care: The Framingham Heart Study*. *Circulation*, 117(6), 743–753.
- Ghorbani, A., & Zou, J. (2019). *Interpretable Machine Learning for Health Care*. *Nature Medicine*, 25(1), 24–29.
- Krittanawong, C., et al. (2020). *Machine Learning Prediction in Cardiovascular Medicine: A Systematic Review*. *Journal of the American College of Cardiology*, 77(12), 167–180.

3. Libraries & Frameworks

- Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. *Journal of Machine Learning Research*, 12, 2825–2830.
- Flask. *Flask Documentation*. Available at:
<https://flask.palletsprojects.com/>
- Pandas. *Pandas Documentation*. Available at: <https://pandas.pydata.org/>
- NumPy. *NumPy Documentation*. Available at: <https://numpy.org/>

APPENDIX-A

CODING

Handling outliers

Capped extreme values in health measures (e.g., BP, BMI, heart rate, etc.) using the IQR method to reduce outlier impact.

```
def cap_outliers(series):  
  
    Q1 = series.quantile(0.25)  
    Q3 = series.quantile(0.75)  
    IQR = Q3 - Q1  
    lower_bound = Q1 - 1.5 * IQR  
    upper_bound = Q3 + 1.5 * IQR  
    return series.clip(lower=lower_bound, upper=upper_bound)
```

Categorical Encoding

```
df_fr=pd.get_dummies(df_fr,columns=['sex','is_smoking','glucose_status','smoking_intensity'])
```

Feature Selection

```
#Dropping of id column as it is unique  
df_fr.drop('id',axis = 1, inplace = True)  
  
# Crezted a copy of df_fr as df_log_reg to preserve the data  
df_log_reg = df_fr.copy()  
  
# Defined X as all feature columns (excluding 'IsatRisk') and y as the target column 'IsatRisk'.  
  
X = df_log_reg.drop('IsatRisk', axis=1) # Features  
  
y = df_log_reg['IsatRisk'] # Target variable  
  
# Saved all feature column names from X into the variable 'columns'.  
columns = X.columns
```

Data Splitting

```
# Split data into train and test sets (33% test) while keeping CHD class
balance using stratify..
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33,
random_state=42, stratify = y)
```

Data Scaling

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

Handling Imbalanced Dataset

```
from imblearn.over_sampling import SMOTE
```

```
smote =SMOTE()
```

```
# fit predictor and target variable smote = SMOTE(random_state=42)
```

```
X_train_res, y_train_res = smote.fit_resample(X_train_scaled, y_train)
```

```
print('Original training shape:', X_train.shape, y_train.shape)
```

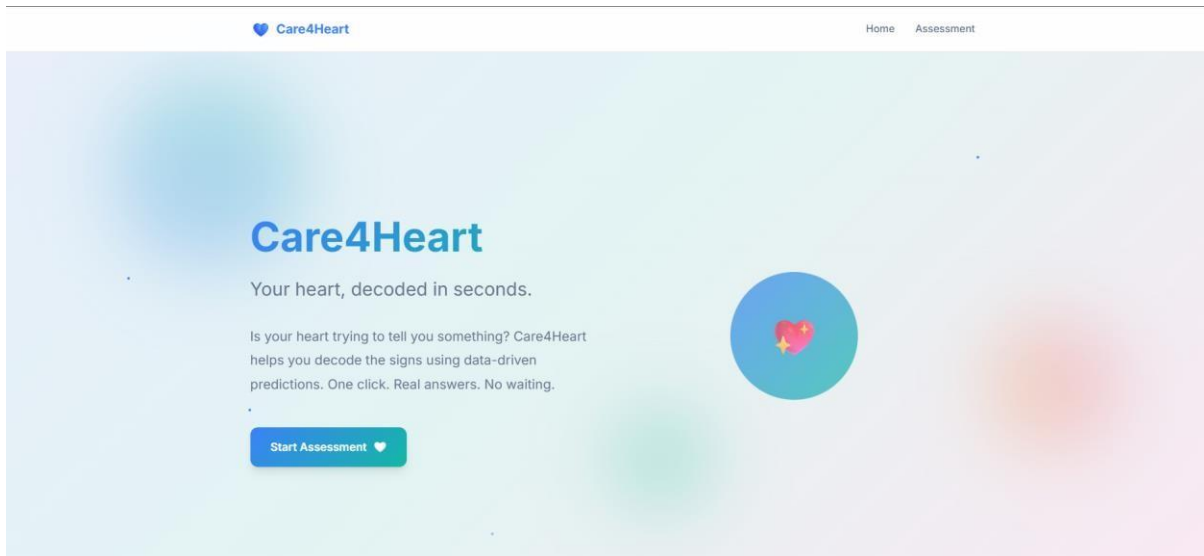
```
print('Resampled training shape:', X_train_res.shape, y_train_res.shape)
```

```
print("Before SMOTE:", y_train.value_counts())
```

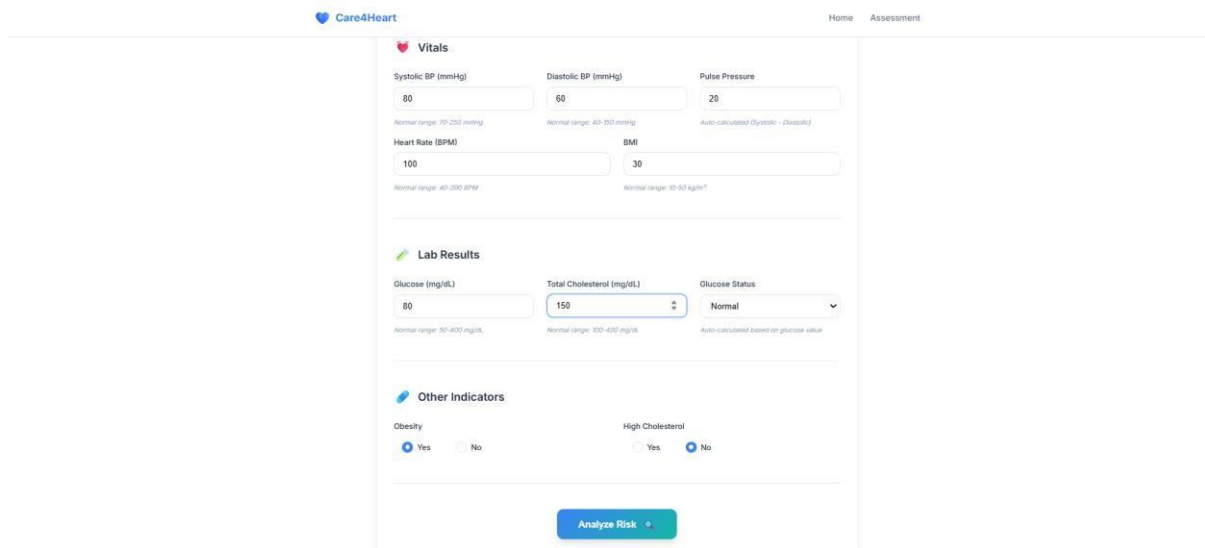
```
print("After SMOTE:", y_train_res.value_counts())
```

APPENDIX-B

WEBSITE DEMONSTRATIONS



Fig(B.1)Dashboard

The image shows the Care4Heart assessment form. It is a web-based form with a light blue and white background. The form is divided into several sections. The 'Vitals' section includes input fields for 'Systolic BP (mmHg)' (80), 'Diastolic BP (mmHg)' (60), 'Pulse Pressure' (20), 'Heart Rate (BPM)' (100), and 'BMI' (30). Below these fields are normal range indicators: 'Normal range: 10-250 mmHg' for Systolic BP, 'Normal range: 40-100 mmHg' for Diastolic BP, 'Auto-calculated (Systolic - Diastolic)' for Pulse Pressure, 'Normal range: 40-200 BPM' for Heart Rate, and 'Normal range: 10-50 kg/m²' for BMI. The 'Lab Results' section includes input fields for 'Glucose (mg/dL)' (80), 'Total Cholesterol (mg/dL)' (150), and a dropdown for 'Glucose Status' (Normal). Below these fields are normal range indicators: 'Normal range: 50-400 mg/dL' for Glucose, 'Normal range: 100-400 mg/dL' for Total Cholesterol, and 'Auto-calculated based on glucose value' for Glucose Status. The 'Other Indicators' section includes radio buttons for 'Obesity' (Yes selected) and 'High Cholesterol' (No selected). At the bottom of the form is a blue button with the text 'Analyze Risk' and a small heart icon.

Fig(B.2)Form

Care4Heart

HomeAssessment

Health Assessment

Please provide your health information for accurate risk prediction

Demographics

Age

19

Age ranges: 18-100 years

Sex

☐ Male
☒ Female

Education Level

College

Lifestyle

Do you smoke?

☐ Yes
☒ No

Cigarettes per day

0

Enter 0-100 cigarettes per day

Smoking Intensity

Non-Smoker

Auto-calculated based on cigarettes per day

Medical History

Diabetes

☒ Yes
☐ No

Blood Pressure Medication

☐ Yes
☒ No

Hypertension

☐ Yes
☒ No

Previous Stroke

☐ Yes
☒ No

Fig(B.3)Form

Care4Heart

HomeAssessment

Other Indicators

Obesity

☐ Yes
☐ No

High Cholesterol

☐ Yes
☐ No

Analyze Risk

✓

Cardiovascular Disease Risk: Low Risk

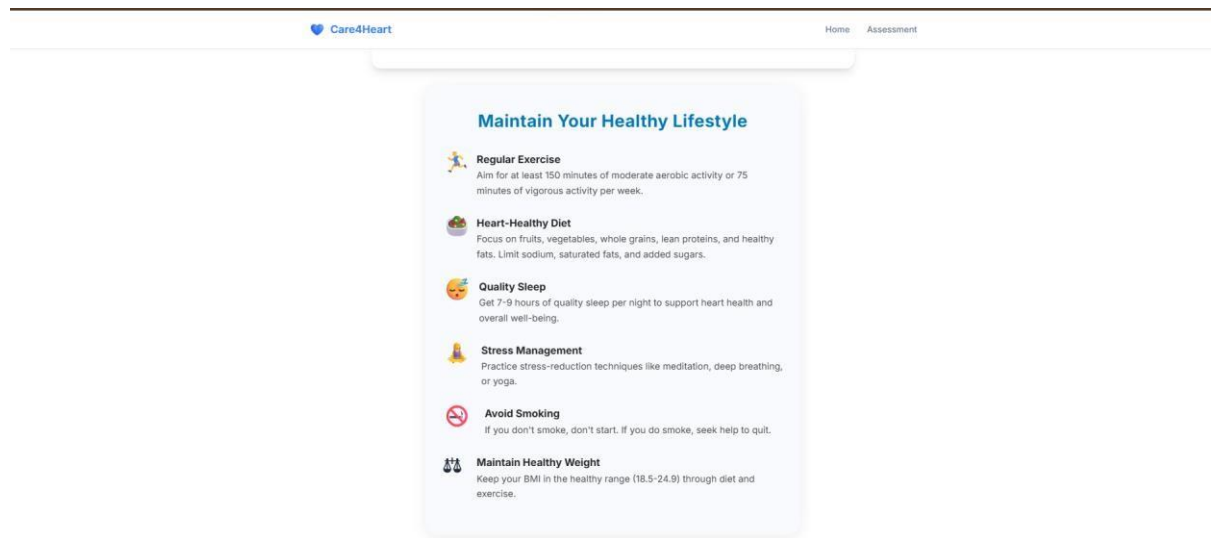
Great job! You are at LOW risk. Maintain a healthy lifestyle to keep your heart strong.

Risk Score: 1%

Maximize Health

Share Results

Fig(B.4)Showing low risk result



Fig(B.5)showing the lifestyle tips

Health Assessment

Please provide your health information for accurate risk prediction

Demographics

Age: 64 (Age range: 18-120 years)

Sex: ☒ Male ☐ Female

Education Level: Graduate

Lifestyle

Do you smoke? ☒ Yes ☐ No

Cigarettes per day: 20 (Error: 0-100 cigarettes per day)

Smoking Intensity: Moderate Smoker (Auto-calculated based on cigarettes per day)

Medical History

Diabetes: ☒ Yes ☐ No

Blood Pressure Medication: ☐ Yes ☒ No

Hypertension: ☒ Yes ☐ No

Previous Stroke: ☒ Yes ☐ No

Fig(B.6)form

Care4Heart

HomeAssessment

Vitals

Systolic BP (mmHg)

160

Normal range: 70-200 mmHg

Diastolic BP (mmHg)

100

Normal range: 40-100 mmHg

Pulse Pressure

60

Auto-calculated (Systolic - Diastolic)

Heart Rate (BPM)

170

Normal range: 40-200 BPM

BMI

30

Normal range: 10-60 kg/m²

Lab Results

Glucose (mg/dL)

250

Normal range: 50-400 mg/dL

Total Cholesterol (mg/dL)

260

Normal range: 50-400 mg/dL

Glucose Status

Diabetes

Auto-calculated based on glucose value

Other Indicators

Obesity

☒ Yes

☐ No

High Cholesterol

☒ Yes

☐ No

Analyze Risk

Care4Heart

HomeAssessment

Other Indicators

Obesity

☐ Yes☒ No

High Cholesterol

☐ Yes☒ No

Analyze Risk

⚠️

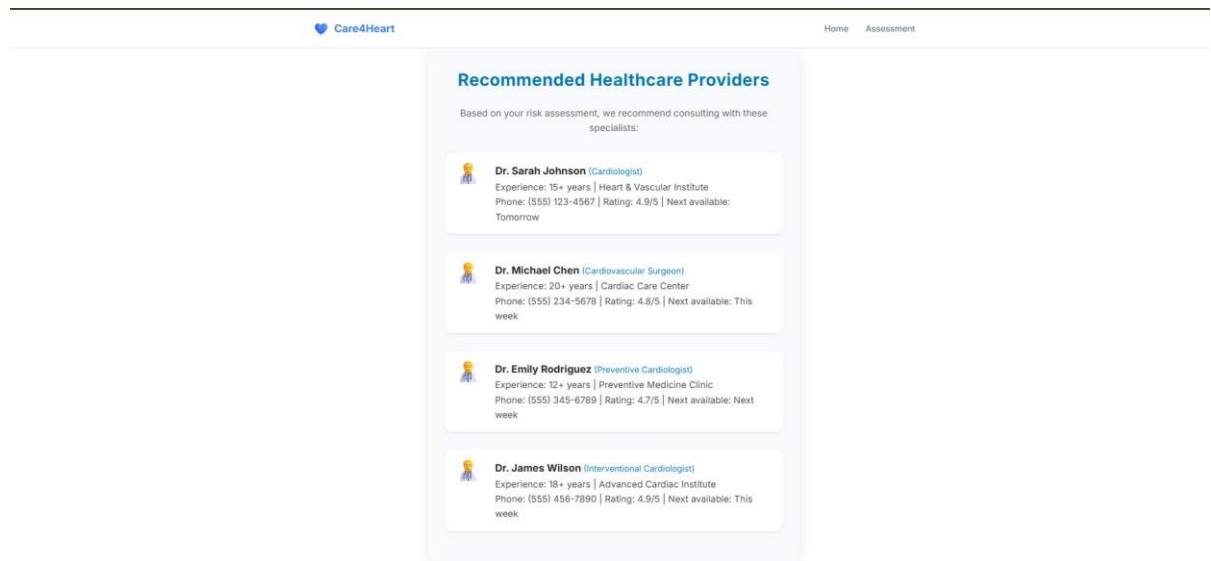
Cardiovascular Disease Risk: High Risk

Based on your profile, you are at HIGH risk for cardiovascular disease. Please consult a cardiologist immediately.

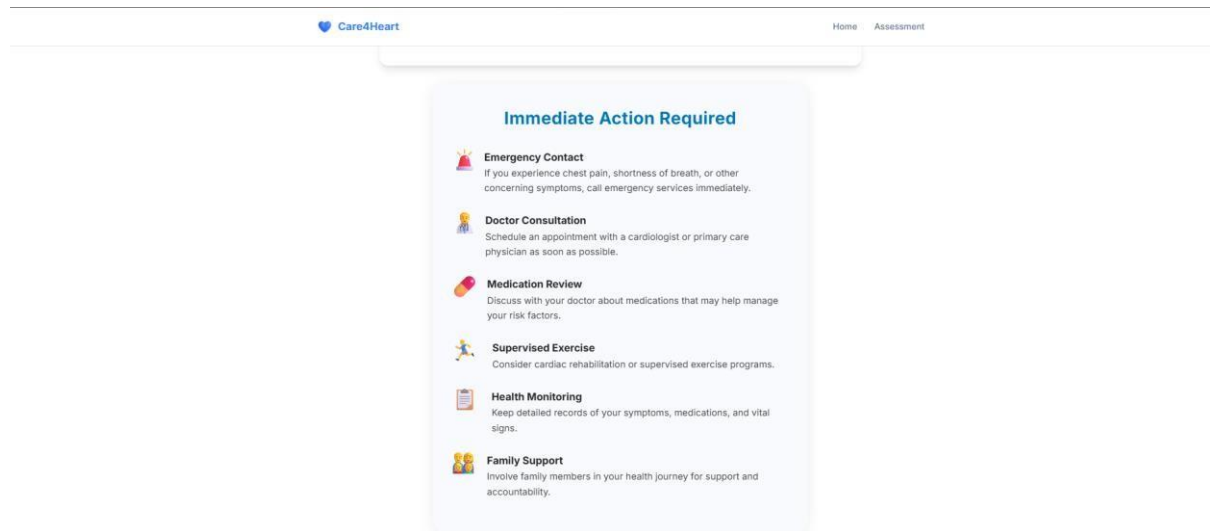
Risk Score: 69%

Consult Doctor

Learn More



Fig(B.9)recommended healthcare providers



Fig(B.10)immediate action required